AGENT 2: Label Template Designer & ZPL Generation (Phases 4–5)

You are continuing development of the GS1 DataMatrix UDI Label Management System. Phases 1–3 are complete. You are responsible for Phases 4–5: the label template designer and ZPL II command generation.

Read MASTER-INSTRUCTIONS.md and review the existing codebase before writing any code.

CRITICAL LABEL FORMAT RULE

LOT MODE: (01) GTIN + (10) Lot + (11) Mfg. NO expiration. **SERIAL MODE:** (01) GTIN + (17) Exp + (11) Mfg + (21) Serial. WITH expiration.

Encoding order (DataMatrix): fixed-length AIs first, variable-length last. **HRI display order** (printed text): GTIN → identifier → date(s).

These are intentionally different. ZPL `^BX` uses encoding order. ZPL `^FD` text commands use display order.

EXISTING CODEBASE

- React + TypeScript + Vite + Tailwind frontend
- Express + TypeScript backend with JSON file storage (`JsonStore<T>`)
- All stores: userStore, productStore, templateStore, printerStore, printJobStore, serialCounterStore, auditLogStore
- Auth, product selector (39 pre-seeded products), GS1 encoding (lot/serial aware)
- Label data form with auto-generated lots, auto-suggested serials
- Live preview with layout engine (stacked vs side-by-side), size slider, text alignment
- Barcode preview with bwip-js

FILES YOU WILL CREATE

```
client/src/components/LabelDesigner/
├── LabelDesigner.tsx
├── DesignerCanvas.tsx
├── ElementToolbar.tsx
├── ElementProperties.tsx
├── TemplateManager.tsx
```

```
├── TemplateList.tsx
├── TemplateImportExport.tsx
├── elements/
│   ├── types.ts
│   ├── DataMatrixElement.tsx
│   ├── HriTextElement.tsx
│   ├── StaticTextElement.tsx
│   ├── LineElement.tsx
│   └── RectangleElement.tsx

client/src/types/template.types.ts

server/src/routes/templates.routes.ts
server/src/controllers/templates.controller.ts
server/src/services/zpl.service.ts
server/src/routes/labels.routes.ts        # NEW: PDF/PNG export endpoints
server/src/services/labelExport.service.ts  # NEW: PDF/PNG generation
```

Install: `cd client && npm install react-konva konva`

PHASE 4: LABEL TEMPLATE DESIGNER

Step 4.1 — Element Type Definitions

`client/src/types/template.types.ts`:

```typescript
```

```typescript
export type MeasurementUnit = 'mm' | 'in';
export type TextAlign = 'left' | 'center' | 'right';  // NEW

export interface BaseElement {
  id: string;
  type: ElementType;
  x_mm: number;
  y_mm: number;
  rotation: 0 | 90 | 180 | 270;
  locked: boolean;
}

export type ElementType = 'datamatrix' | 'hri_text' | 'static_text' | 'line' | 'rectangle';

export interface DataMatrixElementData extends BaseElement {
  type: 'datamatrix';
  moduleSize: number;
}

export interface HriTextElementData extends BaseElement {
  type: 'hri_text';
  fontSize: number;
  fontFamily: string;
  lineSpacing: number;
  textAlign: TextAlign;  // NEW: left, center, right
}

export interface StaticTextElementData extends BaseElement {
  type: 'static_text';
  text: string;
  fontSize: number;
  fontFamily: string;
  bold: boolean;
  textAlign: TextAlign;  // NEW
}

// LineElementData, RectangleElementData — same as original spec

export interface LabelTemplate {
  id: string;
  templateGroupId: string;
  version: number;
  isCurrent: boolean;
```

```
  name: string;
  description?: string;
  widthMm: number;
  heightMm: number;
  marginMm: number;
  dpi: number;
  contentScale: number;    // NEW: 0.5–2.0, default 1.0
  elements: LabelElement[];
  createdBy: string;
  createdAt: string;
}

export const LABEL_PRESETS = [
  { name: '1" × 1" (square small)', widthMm: 25.4, heightMm: 25.4 },  // DEFAULT
  { name: '2" × 1" (standard)', widthMm: 50.8, heightMm: 25.4 },
  { name: '1.5" × 0.75" (small)', widthMm: 38.1, heightMm: 19.05 },
  { name: '2" × 2" (square)', widthMm: 50.8, heightMm: 50.8 },
  { name: '3" × 2" (large)', widthMm: 76.2, heightMm: 50.8 },
  { name: '4" × 2" (wide)', widthMm: 101.6, heightMm: 50.8 },
  { name: '4" × 3" (extra large)', widthMm: 101.6, heightMm: 76.2 },
  { name: 'Custom', widthMm: 0, heightMm: 0 },
] as const;

// Unit helpers
export function mmToInches(mm: number): number {
  return Math.round((mm / 25.4) * 1000) / 1000;
}
export function inchesToMm(inches: number): number {
  return Math.round(inches * 25.4 * 100) / 100;
}
```

Step 4.2 — Designer Canvas

`DesignerCanvas.tsx` using react-konva:

- Stage sized to label dimensions × display scale
- White background = label surface
- Dashed inner rectangle = margin safe zone
- Draggable elements, boundary warnings (red outside label, orange outside margin)
- Arrow key nudge: 0.25mm, Shift+Arrow: 1mm
- Snap-to-grid toggle (0.5mm)
- Zoom: 50%–200%

- Unit toggle: mm / inches (display only, stored as mm)

Square labels: Canvas renders as square. Elements should auto-arrange for stacked layout by default.

Step 4.3 — Element Toolbar

Add DataMatrix (max 1), Add HRI Text (max 1), Add Text, Add Line, Add Rectangle.
Grid toggle, Undo/Redo (50 states), Zoom, Unit toggle.
HRI compliance warning banner if no HRI text element exists.

Step 4.4 — Element Properties Panel

For HRI Text — includes:

- Font size, Font family
- Line spacing
- **Text Align:** dropdown with Left / Center / Right ← NEW
- Position, Rotation, Lock, Delete

For Static Text — includes Text Align as well.

Step 4.5 — Template Manager & API

All operations use `templateStore` (JsonStore reading `data/templates.json`).

API routes:

- `GET /api/templates` — current versions only (isCurrent=true)
- `GET /api/templates/:id` — single
- `POST /api/templates` — create new (version=1)
- `PUT /api/templates/:id` — versioned update (old→isCurrent=false, new→version+1)
- `POST /api/templates/:id/duplicate` — copy with new templateGroupId
- `DELETE /api/templates/:id` — admin only, prevent if referenced by print jobs
- `GET /api/templates/:id/versions` — all versions
- Export/Import endpoints (same as original spec)

Default templates seeded on first run (1"×1" square is the default):

1. "Square 1×1 inch" (25.4×25.4mm) — DataMatrix + HRI text, stacked layout
2. "Standard 2×1 inch" (50.8×25.4mm) — DataMatrix + HRI text, side-by-side
3. "Small 1.5×0.75 inch" (38.1×19.05mm) — DataMatrix + HRI text

4. "Large 3×2 inch" (76.2×50.8mm) — DataMatrix + HRI + static text

Step 4.6 — Integration with Label Form

Template selector dropdown on the label creator page. Default selection: Square 1×1 inch.
Composite preview uses the layout engine from Phase 3.

Step 4.7 — Test Phase 4

Same as original spec, plus:

- Default template is 1"×1" square
- HRI text alignment property saves and loads correctly
- Text alignment visible in preview and designer

PHASE 5: ZPL GENERATION & LABEL EXPORT

Step 5.1 — ZPL Service

`server/src/services/zpl.service.ts`:

```typescript
```

```
function mmToDots(mm: number, dpi: number): number {
  return Math.round(mm * (dpi / 25.4));
}

function fontSizeToDots(fontSize: number, dpi: number): number {
  return Math.round(fontSize * dpi / 72);
}

export function generateZpl(input: ZplInput): string {
  // ^XA ... ^XZ
  // DataMatrix: ^BX with ENCODING order
  // HRI: ^FD text lines in DISPLAY order with text alignment
  // Text alignment: left = ^FO+^A+^FD, center/right = ^FO+^FB{width},{lines},0,{justify}+^FD
  //   justify: L=left, C=center, R=right
}

export function buildZplGs1String(data): string {
  // LOT: (01)GTIN(11)YYMMDD(10)LOT — NO AI 17
  // SERIAL: (01)GTIN(17)YYMMDD(11)YYMMDD(21)SERIAL — WITH AI 17
  // GS separator = ~d029
}

export function generateBatchZpl(input, serials, copiesPerLabel): string
export function generateLotBatchZpl(input, copies): string
```

Step 5.2 — Label Export Service (NEW)

`server/src/services/labelExport.service.ts`:

Single label PNG:

- Generate ZPL → send to Labelary API → return PNG
- Filename: `{gtin}-{lot or serial}-label.png`

Single label PDF:

- Generate label image → create PDF with page size = label dimensions (NOT letter size)
- e.g., 25.4mm × 25.4mm page for 1"×1" label
- Filename: `{gtin}-{lot or serial}-label.pdf`

Batch PDF (serial mode):

- One page per serial, each page = label size

- Max 500 labels per batch
- Filename: `{gtin}-batch-{firstSerial}-{lastSerial}.pdf`

Batch PNG ZIP (serial mode):

- One PNG per serial in a ZIP file
- Filenames: `{gtin}-{serial}.png`
- Max 500 labels

LOT batch PDF:

- All pages identical (same lot number)
- Max 500 copies

Step 5.3 — Export API Routes

`server/src/routes/labels.routes.ts`:

- `POST /api/labels/export-png` — single label PNG
- `POST /api/labels/export-pdf` — single label PDF
- `POST /api/labels/export-batch-pdf` — batch PDF
- `POST /api/labels/export-batch-png` — batch PNG ZIP

Step 5.4 — ZPL Preview API

- `POST /api/zpl/generate` — returns ZPL string
- `POST /api/zpl/preview-image` — sends ZPL to Labelary, returns PNG
- `POST /api/zpl/batch-generate` — returns batch ZPL

Step 5.5 — Frontend Export Buttons

On label creator page:

- "Save as PDF" button
- "Save as PNG" button
- "Download ZPL" button
- All disabled until form is complete and confirmation checkbox is checked
- Batch export buttons show total count and warn if > 500

Step 5.6 — Test Phase 5

Same as original spec, plus:

- LOT mode PDF: page size = label dimensions, 3 HRI lines, no expiration
- SERIAL mode PDF: 4 HRI lines, with expiration
- Batch PDF for 5 serials: 5 pages, each correct
- Text alignment reflected in ZPL output (^FB command for center/right)
- Square label ZPL: ^PW and ^LL are equal